ITSD 85 Individual Contribution Log (Solo Developer)

Course: ITSD 85 – Special Topics in Software Development

Laboratory Title: Mobile Game Development with Level Progression and Offline/Online Multiplayer

Game Title: Dead Block: Survival

Student Name: Joenel Pacal

Student ID / Email: pacaljoenel08@gmail.com

Role: Solo Developer (100% Architecture, Gameplay, AI, UI, Audio, Networking, Testing & CI/CD)

GitHub Account Username: @yurizz-crypto / @yuri-salise

Repository: https://github.com/yuri-salise/boxhead-survival

Primary Test Device: Samsung Galaxy A56 (Android 15 / One UI 7)

Date Submitted: November 10, 2026

---

1. Summary of Individual Contributions (100% Project Scope)

As a solo developer, I designed, implemented, and empirically verified all components of the project:

Contribution Area	Specific Features / Modules Implemented	Associated Files
Core Gameplay & AI	Built player twin-stick locomotion, tactical dash invulnerability, and offline `BRAVO BOT` companion with autonomous kiting and medical heal logic.	`scripts/player.gd`, `scripts/weapon_controller.gd`
Enemy Engineering	Implemented 14 specialized zombie variants including the apex boss `Necro` (passive healing aura, witch coven), Boomer explosions, and Spitter acid mortars.	`scripts/zombie_basic.gd`, `scenes/actors/zombie_basic.tscn`
Progressive Level Design	Designed Sectors 1–5 procedural chunk generation, vehicle roadblocks, toxic puddles, and blackout plaza generator defense stations.	`scripts/arena_environment.gd`, `scenes/levels/level_01.tscn` .. `05.tscn`
Multiplayer Networking	Built host-authoritative ENet LAN synchronization at 12.5 Hz, client hit verification, squadmate revive, and graceful disconnect recovery.	`autoload/network_manager.gd`, `scenes/ui/lobby.gd`

Contribution Area	Specific Features / Modules Implemented	Associated Files
UI & Audio	Created high-contrast survival UI, virtual joysticks, 21 synthesized 16-bit sound effects, and looping ambient music.	`autoload/audio_manager.gd`, `scenes/ui/`
Testing & CI/CD	Authored headless test suites (23 Godot suites + Node.js suite) and configured GitHub Actions automated deployment.	`tests/`, `.github/workflows/ci-cd.yml`
Academic Paper	Wrote all 8 sections and the Appendix of the IEEE conference paper, analyzing the Samsung Galaxy A56 hardware metrics and Likert evaluation.	`docs/paper/ieee_paper.md`, `docs/paper.pdf`

---

2. Verified Git Commit Log

Note: Below are the direct permanent commit URLs verifying my individual authorship on GitHub:

1. **Commit 0082688:** `feat: mobile UI/UX redesign, shader coordinate fix, death retry fix, and 4-player squad co-op with teammate revive`

Description: Implemented full touch-screen control ergonomics, responsive layout scaling, and cooperative squadmate revival state mechanics.

2. **Commit fcad6cb:** `feat(phase5): implement 2-player LAN co-op with NetworkManager, Lobby UI, and host-authoritative replication`

Description: Created quantized 12.5 Hz state broadcast, remote client hit validation, and local network peer discovery.

3. **Commit eaef5fc:** `feat(level): implement unique arena layouts for Levels 1-5 matching GDD`

Description: Built procedural obstacle geometry, environmental hazards, interactive defense turrets, and diesel fuel truck mechanics.

4. **Commit 025aec4:** `feat(enemies): add necro boss variant with 7-witch coven escort`

Description: Added custom sound triggers, passive regeneration aura, and witch minion escort formation hierarchy.

5. **Commit 27d4511:** `fix(multiplayer, ui): sync interactable states across peers, add scrollable responsive modals`

Description: Synchronized generator and turret illumination across all connected clients and resolved small-screen modal clipping.

---

3. Reflection on Solo Development & Technical Challenges Overcome

During this development lifecycle, executing a complete 5-level mobile multiplayer game as a solo developer required strict architectural discipline and disciplined time management. One of the most significant technical hurdles was mobile network desynchronization, which I resolved by eliminating naive peer-to-peer client authority and enforcing strict host-authoritative state resolution with dead reckoning. Working within Godot 4 Standard GDScript and the Android `gl_compatibility` renderer allowed me to maintain 55–60 FPS on my primary physical test device (Samsung Galaxy A56) while shipping a feature-complete, zero-dependency offline and online experience.